



Ferrum Bellum

Documentación Técnica

Autores:

Daniel Durán García – 21.493.398-5

Martín Castillo Torrejón – 21.484.992-5

Fecha: 20 de junio 2025

Asignatura: Estructura de Datos

Carrera: Ingeniería en Tecnología de Información

Institución: Universidad Católica del Norte



Contenido

Introducción	2
Tecnologías Utilizadas	3
Estructura del Proyecto	4
Modo de Juego	5
Inteligencia artificial	6
Requisitos del sistema	6
Compilación y Ejecución	7
Compilación	7
Ejecución	7
Descripción del código	8
Diseño Técnico e Implementación de Algoritmos.....	10
Estructura de datos.....	10
IA avanzada (Dificultad “Difícil”)	11
Análisis de Tiempos de Decisión de la IA (Dificultad Difícil)	12
Justificación de la Función Heurística.....	13
Conclusión	14



Introducción

Este documento describe la implementación técnica del proyecto Ferrum Bellum, un juego por turnos en C++ con interfaz gráfica desarrollada en SFML. El juego está diseñado para ejecutarse en sistemas Windows usando el entorno de desarrollo MSYS2 con el perfil ucrt64.

Tecnologías Utilizadas

- Lenguaje: C++
- Librería gráfica: SFML 2.6.1 (versión dinámica)
- Entorno de compilación: MSYS2 (UCRT64)
- Compilador: g++
- IDE recomendado: Visual Studio Code

Estructura del Proyecto

A continuación, se detalla la estructura del proyecto Ferrum Bellum, indicando la función principal de cada carpeta o archivo.

Proyecto:

- .vscode → Archivos de configuración de Visual Studio Code.
- Dominio → Contiene archivos .h y .cpp de las Clases Tanques y Nodos.
- Fuentes → Fuente utilizada para los textos en la interfaz gráfica.
- Imágenes → Recursos gráficos usados (Sprites).
- Lógica → Contiene el archivo main.cpp, con la lógica del juego.
- Sonidos → Contiene los archivos de música usados en el juego.
- README.md → Contiene información de los integrantes (nombre, rut, carrera).
- app.exe → Ejecutable generado tras compilar el proyecto.
- Documentacion_Ferrum_Bellum.pdf
- (Otros archivos) → Archivos diversos.

Modo de Juego

El flujo general del juego se desarrolla en las siguientes etapas:

1. Pantalla de inicio

El jugador comienza en una pantalla de inicio, donde deberá presionar cualquier tecla.

2. Menú Principal

Se despliega un menú con las siguientes opciones:

- Jugar: Inicia el proceso del juego
- Configuraciones: permite acceder a:
 - Ajuste del volumen.
 - Cambio de resolución de pantalla.
 - Restaurar valores por defecto.
 - Volver al menú anterior.
- Salir: Cierre el juego de forma inmediata.

3. Selección de dificultad

El jugador elige el nivel de dificultad de la inteligencia Artificial: Fácil, Medio o Difícil.

4. Selección de Tanques y Posicionamiento Inicial.

El jugador selecciona tres tanques de diferentes tipos (Ligeros, Mediano, Pesado) y define sus posiciones iniciales en el tablero.

5. Fase de combate

Se inicia el bucle de combates por turnos. En cada ronda, tanto el jugador como la IA eligen sus acciones (moverse o disparar), las cuales se ejecutan de formas simultánea.

6. Resultado de la Partida

Una vez finalizado el combate, se muestra una pantalla de victoria o derrota. AL presionar la tecla Esc, se regresa al menú principal, permitiendo reiniciar una nueva partida.

Inteligencia artificial

Nivel	Descripción
Nivel 1 (Básico)	Disparo basado solo en la posición del jugador. No considera vida ni movimientos. (Probabilidad simple).
Nivel 2 (Intermedio)	Considera posición y distancia. Decide si moverse o disparar. (Probabilidad + lógica simple).
Nivel 3 (Avanzado)	Considera todos los factores del juego. Implementa el algoritmo Minimax con poda alfa-beta.

Requisitos del sistema, Compilación y Ejecución

Requisitos del sistema

- Tener instalado MSYS2
- Instalar la librería SFML:
`pacman -S mingw-w64-ucrt-x86_64-sfml`

Compilación

Para compilar el proyecto, asegúrate de estar en la raíz del repositorio y ejecuta el siguiente comando en la terminal de MSYS2:

```
C:/msys64/ucrt64/bin/g++.exe -g Logica/main.cpp
Dominio/Tanques/Tanque.cpp Dominio/Tanques/TanquePesado.cpp
Dominio/Tanques/TanqueLigero.cpp Dominio/Tanques/TanqueMediano.cpp
Dominio/Nodos/NodoSistema.cpp -I C:/msys64/ucrt64/include -o
app.exe -L C:/msys64/ucrt64/lib -lsfml-graphics -lsfml-window -
lsfml-system -lsfml-audio -lopengl32 -lfreetype -lwinmm -lgdi32
```

Ejecución

Una vez que la compilación se haya realizado exitosamente y el archivo app.exe esté disponible, puedes ejecutar el juego con el siguiente comando:

```
./app.exe
```


Descripción del código

A continuación, se describe la funcionalidad de los principales componentes del código fuente del proyecto.

1. Estructura de Archivos y Módulos

- main.cpp
 - Orquesta el flujo: menús, carga de recursos, bucle de juegos, limpieza.
- Carpeta Dominio (Tanque.h/ .cpp, NodoSistema.h/ .cpp)
 - Clases Tanque, TanqueLigero, TanqueMediano, TanquePesado: encapsulan atributos (vida, daño, movimiento) y métodos de clonación y probabilidad de impacto.
 - Clase NodoSistema: nodo de lista doblemente enlazada que modela cada casilla del tablero.

2. Funciones de Tablero y Movimiento

- crearTablero(), agregarPosicion(): construcción del tablero 5 x 5 con tipos de terreno.
- moverse(), disparar(): ejecutan las acciones sobre el tablero real.
- moverseSimulado(), dispararSimulado(): versiones para “simular” sin afectar el estado real.

3. Interfaz Gráfica y Menús

- mostrarPantallaInicio(), mostrarMenuPrincipal(), mostrarMenuDificultad(), mostrarMenuSeleccionTanquesJugador(): dibujan y gestionan de la UI de menús.
- menuAccionesJugadorSimultaneo(): dibuja tablero de juego, tanques IA y jugador, acciones (moverse o disparar) y coordina los movimientos simultáneos de jugador e IA.
- desplegarTablero(), mostrarMensajeFinal(): renderizado de tablero, tanques, explosiones y mensajes de fin de partida.

4. IA y Toma de Decisiones

- Generación de Estados: generarHijosAvanzados() crea copias de estado para evaluar movimientos y disparos.
- Heurísticas Rápida: evaluarAmenzasTerritoriales() para pre-filtrar hijos.
- Evaluación Completa: evaluarEstadoAvanzado() combina vida, terreno, distancia, control posicional, movilidad y patrones.



- Minimax: minimaxAvanzado() con poda alfa-beta para profundizar hasta profundidadBase adaptativa.
- Orquestación: encontrarMejorJugadaAvanzada() mide tiempos y actualiza memoria táctica.

Diseño Técnico e Implementación de Algoritmos

En esta sección se explica cómo se ha estructurado el motor del juego y cómo se implementan los diferentes algoritmos, con especial hincapié en la IA avanzada.

Estructura de datos

- Lista doblemente enlazada (NodoSistema)
Representa el tablero de 5 x 5; cada nodo almacena coordenadas, tipo de terreno y puntero al tanque (si lo hay).
- Pilas de tanques (std::stack<Tanque*>)
Se utiliza para almacenar los tanques del y de la IA en orden de creación.
- EstadoJuego
Clona tablero y tanques para simular movimientos: contiene punteros a las copias de los tanques, el turno actual y la acción planificada.

Flujo Principal

1. main()
 - Ajusta ventana a la resolución seleccionada.
 - Carga recursos (fuente, texturas, música)
 - Muestra menús: inicio → principal → dificultad → selección tanques → combate.
 - Al terminar libera memoria de tanques.
2. Funciones de Interfaz
 - mostrarPantallaInicio(), mostrarMenuPrincipal(), mostrarMenuDificultad(), mostrarMenuSeleccionTanquesJugador().
3. Generación del Tablero
 - crearTablero(): crea los 25 nodos con terreno {planicie, bosque, montaña}.
 - agregarPosicion(): añade nodos al final de la lista.
4. Acciones de Tanques
 - moverse(), disparar(): actualizan estado del tablero real.
 - moverseSimulado(), dispararSimulado(): idénticas pero copias para IA.

IA avanzada (Dificultad “Difícil”)

- Minimax con poda Alfa-Beta (minimaxAvanzado): produce hasta 3 movimientos y 2 disparos más prometedores, ordenados por una evaluación rápida (amenazas territoriales).
- Evaluación heurística (evaluarEstadoAvanzado): combina diez factores (vida, terreno, distancia, control posicional, movilidad, experiencia de la IA, entre otros.) en un valor escalar.
- Poda Alpha-Beta: detiene ramas que no mejoran la mejor opción conocida, reduciendo drásticamente el espacio de búsqueda.

Análisis de Tiempos de Decisión de la IA (Dificultad Difícil)

Se midió el tiempo que tarda la IA en encontrarMejorJugadaAvanzada():

Contexto	Tiempo (ms)
Primera jugada	56
Máximo en igualdad	426
Media en igualdad	380
Media en ventaja (IA)	100
Media en desventaja	150

- Primera jugada: menor carga de patrones y menor profundidad adaptativa.
- Igualdad de vida: profundidad base y generación completa de estados, por eso valores medios altos (~380ms).
- Ventaja de vida: IA ajusta profundidad a +1, genera menos hijos y poda mas agresiva, reduciendo el tiempo (~100ms).
- Desventaja: IA profundizada +2 y evalúa más caminos defensivos, por eso sube a ~150ms.

Justificación de la Función Heurística

La función evaluarEstadoAvanzado(estado) asigna puntos en base a factores ponderados:

1. Diferencia de vida ($\text{vida} \times 1000$)
 - Un gran multiplicador porque es condición principal de victoria/derrota
2. Bonificación por vida propia ($\text{vidaIA} \times 200$)
 - Refuerza mantener tanques con alta vida.
3. Penalización por vida enemiga ($-\text{vidaJugador} \times 180$)
 - Fomenta reducir la vida del rival.
4. Ventaja de terreno ($(\text{terrenoIA} - \text{terrenoJugador}) \times 50$)
 - Cada tipo de terreno aporta un valor extra, midiendo cuan favorable es la casilla.
5. Evaluación territorial avanzada ($\text{evaluarAmenazasTerritoriales} \times 1$)
 - Control de celdas adyacentes: suma 3 puntos por celda de ventaja y 8 puntos por rutas de escape, equilibrando agresión y seguridad.
6. Control posicional dinámico ($+(\text{centroJugador} - \text{centroIA}) \times 25$)
 - Prioriza ocupar el centro cuando IA domina, o esquinas si pierde.
7. Distancia estratégica
 - Dominando ($-(|\text{dist}-2| \times 20)$): busca distancia optima de ataque a 2 celdas.
 - Ligera ventaja ($-\text{dist} \times 15$): acércate
 - Empate ($+30$ si $\text{dis}=2$): controla distancia media.
 - Desventaja ($+\text{dist} \times 10$ ó 25): incrementar distancia a rivales peligrosos.
8. Exposición a ataques (± 150 ó -300)
 - Si IA está a distancia 1 y en desventaja, penaliza fuertemente para evitar suicidios; si domina, premia el ataque cercano.
9. Movilidad ($(\text{movilidadIA} - \text{movilidadJugador}) \times 40$)
 - Aprecia cuantas opciones de desplazamiento quedan.
10. Experiencia acumulada ($+5 \times \text{exitos}$ y $-2 \times \text{fallos}$)
 - Ajusta la IA para “aprender” de disparos exitosos o fallidos.

Conclusión

El proyecto Ferrum Bellum integra de forma coherente estructuras de datos dinámicas, un motor de IA basado en Minimax con poda Alfa-Beta y una interfaz gráfica fluida desarrollada con SFML. Gracias a la modularidad del código (clases de dominio, manejo de memoria, generación de tablero y capas de simulación), se logra un diseño sólido y fácil de mantener. La IA de nivel “Difícil” demuestra un balanceo efectivo entre complejidad táctica y rendimiento, tomando decisiones en promedio por debajo de 400 ms y adaptando su profundidad de búsqueda en función de la ventaja o desventaja en el combate. La función heurística, fundamentada en diez criterios ponderados (vida, terreno, combate territorial, movilidad, entre otros), permite a la IA priorizar correctamente el ataque, la defensa y el posicionamiento en el tablero. En conjunto, Ferrum Bellum fue un gran proyecto para la aplicación de conceptos de estructuras de datos y algoritmos, dejando una base sólida para futuras implementaciones.